Enterprise Architect MCP — Tool Cheatsheet

Server `enterprise-architect` (`MCP3.exe`), launched with `-enableEdit -setTimeout 60` .
Author: Josef Klesa (klesajos)

★ = editing tool (present only because of `-enableEdit`). Deletion is possible **for connectors/messages only** — back up via a baseline before edits. IDs are numeric and stable (`ElementID`, `DiagramID`, `ConnectorID`, `PackageID`, `AttributeID`, `OperationID`); most tools take them, so discovery (find/get) usually comes first.

📦 Packages

Tool	What it does	Key inputs
`get_root_packages`	Top-level package(s) of the model — your starting point.	–
`get_current_package`	Package currently selected in EA. If an element/diagram is selected, returns its containing package. Use before creating things.	–
`get_packages_information`	Full details of the given packages.	`packageIDs[]`
`find_packages_by_name`	Find packages by name across the model.	`name, exactMatch (def. true)`
★ `create_or_update_package`	Create a package (`packageID:0`) under a parent, or update one (`packageID:≠0`).	`packageInfo{name, owningPackageID, packageID, description}, taggedValues`
★ `clone_package`	Full copy of a package into the same parent.	`packageID`
★ `create_baseline`	**Snapshot/backup** of a package. Returns a baseline ID. Do this before a risky edit.	`packageID`
★ `apply_baseline`	**Revert** a package to a baseline. Auto-refreshes diagrams.	`packageID, baselineID`

🧩 Elements

Tool	What it does	Key inputs
`get_current_elements`	Elements currently selected in EA.	–
`get_elements_information`	Full detail: operations, attributes, child elements and diagrams.	`elementIDs[]`
`find_elements_by_name`	Find elements by name across the model.	`name, exactMatch (def. true)`
`find_element_in_diagrams`	List all diagrams an element appears on.	`elementID`
`select_element_in_browser`	Highlight an element in EA's Browser window.	`elementID`
`select_element_in_diagram`	Select an element in a diagram (opens it if needed).	`elementID, diagramID`
`export_element_linked_documents`	Export elements' linked documents to RTF (`<elementID>.rtf`).	`elementIDs[], pathToExport`
★ `create_or_update_elements`	Create (`elementID:0`) or update (`elementID:≠0`) elements. Supports stereotypes, tagged values and color/border/text formatting (per-diagram or whole model).	`elementInfo[]{name, type, owningPackageID, owningElementID, elementID, stereotypes, taggedValues, elementFormat}`
★ `create_or_update_operations`	Add/update operations (methods) on an element incl. parameters, scope, return type, order.	`elementID, operationInfo[]`
★ `create_or_update_attributes`	Add/update attributes on an element (type, scope, order, stereotype).	`elementID, attributeInfo[]`
★ `clone_elements`	Full copy of elements into the same parent package.	`elementID[]`
★ `import_element_linked_documents`	Import RTF linked documents onto elements (file must be `<elementID>.rtf`).	`elementID[], pathToImport`

🔗 Connectors & Messages

Tool	What it does	Key inputs
`get_current_connector`	Connector selected in the current diagram.	–
`get_connectors_information`	Full detail incl. tagged values.	`connectorIDs[]`
★ `create_or_update_connectors`	Create (`connectorID:0`) or update relationships between elements. Set `sourceEnd`/`targetEnd` (relatedElementID, multiplicity, scope, name), style, color/width, stereotypes, tagged values. Reload the diagram afterwards.	`connectorInfo[]{type, sourceEnd, targetEnd}`
★ `create_or_update_messages`	Create/update messages between lifelines on a **Sequence diagram**. Supports async/return, operation-typed messages, ordering.	`diagramID, messageInfo[] {sourceElementID, targetElementID, order}`

Tool	What it does	Key inputs
★ `delete_connectors_or_messages`	**The only delete tool.** Removes connectors/messages. Reload the diagram afterwards.	`connectorIDs[]`

📐 Diagrams

Tool	What it does	Key inputs
`get_current_diagram`	The active diagram in EA.	–
`get_opened_diagrams`	All currently open diagrams.	–
`get_diagrams_information`	Detail incl. shown elements and connectors. **Use this to analyze a diagram**, not the image.	`diagramIDs[]`
`get_diagram_image`	PNG render of a diagram (visual only, not for analysis).	`diagramID`
`open_diagrams`	Open diagrams in EA.	`diagramIDs[]`
`reload_diagrams`	Refresh diagrams so new elements/connectors show.	`diagramIDs[]`
★ `create_or_update_diagram`	Create (`diagramID:0`) or update a diagram, owned by a package or an element. Type can't change after creation.	`diagramInfo{name, type, owningPackageID, owningElementID, diagramID}`
★ `place_elements_on_diagram`	Place existing elements on a diagram at x/y with width/height. Auto-reloads — no manual reload needed.	`diagramID, placements[]{elementID, x, y, width, height, style}`
★ `layout_connectors`	Auto-layout the connectors in a diagram.	`diagramID`
★ `change_connector_visibility`	Show/hide specific connectors on a diagram.	`diagramID, connectorIDs[], showHide`

Conventions & gotchas

- **Create vs. update** is signaled by the ID field: `0` = create new, non-zero = update existing. Applies to elements, diagrams, packages, connectors, operations, attributes and messages.
- **`type` is immutable** after creation for elements, connectors and diagrams.
- **Stereotyped/profile types** use the fully-qualified name, e.g. `Archimate3::ArchiMate_ApplicationComponent`, `Archimate3::Application` (diagram), `Archimate3::ArchiMate_Serving` (connector).
- **Ownership:** an element/diagram belongs to a package (`owningPackageID`) OR a parent element (`owningElementID`, set the other to 0).
- **Reload after relationship edits:** connectors/messages need `reload_diagrams`; element placement auto-reloads.
- **Colors:** RGB 0–255 per channel; `red:-1` = use default color. `borderWidth`/connector width: -1..3, -1 = default.
- **Scope** values: `Private`, `Protected`, `Package`, `Public`.
- **Linked documents** are RTF; the filename must be `<elementID>.rtf`.
- **Timeout** is 60 s (`-setTimeout 60`); raise toward 600 for heavy operations (EA runs under x64 emulation here).
- **Timeout / "Failed to connect"** almost always means **no project is open in EA** — open the repository first, then retry.

Safe edit workflow

1. `get_current_package` / `find_packages_by_name` → target `packageID`.
2. ★ `create_baseline(packageID)` → keep the returned baseline ID.
3. Make edits (create/update elements, connectors, diagrams…).
4. `reload_diagrams` to verify visually.
5. If something is wrong → ★ `apply_baseline(packageID, baselineID)` to roll back.

> **Beware deletion:** the MCP has no tool to delete packages or elements — the only delete is `delete_connectors_or_messages`. Delete test/unwanted packages and elements manually in EA (right-click → Delete). Name throwaways e.g. `ZZ_*` so they are easy to find.